



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Desarrollo de Aplicaciones Web	Unidad 2	Tema: Protocolo HTTP
	Semana 4	Tutor: Ing. Victoria Castro

Introducción al Protocolo:

El **Protocolo de Transferencia de Hipertexto (HTTP)** es un protocolo de la **capa de aplicación** (según el modelo OSI) que permite la comunicación en la web. Su función principal es la transferencia de recursos, que pueden ser desde documentos HTML hasta imágenes, videos o datos en formato JSON para aplicaciones modernas.

Se define como un protocolo **sin estado (stateless)**, lo que significa que el servidor no guarda ningún dato (estado) entre dos peticiones sucesivas. Para solucionar esto en la web moderna, se utilizan las **Cookies** y los encabezados de estado, permitiendo sesiones de usuario coherentes.

Sección 1:

Orígenes y Evolución Histórica

La historia de HTTP está ligada a la evolución de las capacidades de la red:

- **HTTP/0.9 (1991):** La "versión de una sola línea". Solo permitía el método GET y las respuestas solo podían ser archivos HTML. No existían las cabeceras.
- **HTTP/1.0 (1996):** Introdujo las **cabeceras (headers)**, permitiendo enviar metadatos sobre la petición y la respuesta. Se añadieron los códigos de estado y el soporte para otros tipos de archivos (imágenes, scripts).
- **HTTP/1.1 (1997-1999):** La versión más longeva. Introdujo las **conexiones persistentes** (keep-alive) para evitar abrir una conexión TCP por cada archivo, permitiendo cargar sitios complejos más rápido.
- **HTTP/2 (2015) y HTTP/3 (2022):** Enfocados en la optimización. HTTP/2 introdujo el multiplexado (varias peticiones sobre una sola conexión) y HTTP/3 cambió el transporte de TCP a **QUIC** (basado en UDP) para reducir la latencia.

Sección 2:

Métodos de Petición (Verbos HTTP)

Los métodos indican la acción que se desea realizar sobre un recurso específico. Se dividen principalmente en dos tipos: **Idempotentes** y **No Idempotentes**.

Métodos Principales

GET: Se utiliza exclusivamente para **recuperar datos**. No debe tener efectos secundarios en el servidor (es de solo lectura).

POST: Se utiliza para **enviar entidades** a un recurso específico (por ejemplo, enviar un formulario de registro). A menudo causa un cambio de estado en el servidor.

PUT: Reemplaza todas las representaciones actuales del recurso de destino con la carga útil de la petición. Es **idempotente** (si se envía 10 veces, el resultado final es el mismo).

DELETE: Borra el recurso especificado.

PATCH: Se utiliza para aplicar modificaciones parciales a un recurso. A diferencia de PUT, no reemplaza el recurso completo.

HEAD: Pide una respuesta idéntica a la de una petición GET, pero **sin el cuerpo de la respuesta**. Útil para verificar si un archivo existe antes de descargarlo.

OPTIONS: Se utiliza para describir las opciones de comunicación (permisos de CORS) para el recurso de destino.

Sección 3:

Estructura de los Códigos de Respuesta

Los códigos de estado son números de 3 dígitos que el servidor devuelve para informar al cliente sobre el éxito o fallo de la operación. Se agrupan en 5 clases:

Respuestas Informativas (100-199)

Indican que la petición ha sido recibida y el proceso continúa.

- **101 Switching Protocols:** El servidor acepta cambiar el protocolo (por ejemplo, a WebSockets).

Respuestas Exitosas (200-299)

- **200 OK:** La petición ha tenido éxito.
- **201 Created:** La petición ha tenido éxito y se ha creado un nuevo recurso (común en POST).
- **204 No Content:** La petición tuvo éxito pero no hay contenido que enviar (común en DELETE).

Redirecciones (300-399)

- **301 Moved Permanently:** El recurso ha cambiado de URL para siempre.
- **304 Not Modified:** Indica al navegador que el recurso no ha cambiado y puede usar la copia que tiene en su memoria **caché**.

Errores del Cliente (400-499)

- **400 Bad Request:** El servidor no puede entender la petición por sintaxis inválida.
- **401 Unauthorized:** Es necesario autenticarse para obtener la respuesta.

- **403 Forbidden:** El cliente no tiene derechos de acceso (el servidor sabe quién eres, pero no te deja pasar).
- **404 Not Found:** El servidor no pudo encontrar el recurso solicitado.

Errores del Servidor (500-599)

- **500 Internal Server Error:** Error genérico cuando el servidor no sabe cómo manejar la situación.
- **503 Service Unavailable:** El servidor no está listo para manejar la petición (por mantenimiento o sobrecarga).

Sección 4:

Seguridad y Cifrado: De HTTP a HTTPS

Como explica AWS en su comparativa, la "S" en HTTPS significa **Secure**.

- **Vulnerabilidad de HTTP:** Los datos viajan en texto claro. Cualquier nodo intermedio en la red (un router, un ISP) puede realizar un ataque de *man-in-the-middle* para leer contraseñas o datos bancarios.
- **Cifrado TLS/SSL:** HTTPS utiliza un apretón de manos (**Handshake**) criptográfico. Se intercambian claves públicas y privadas para establecer un canal cifrado.
- **Integridad y Autenticidad:** HTTPS garantiza que el contenido no ha sido modificado en el camino y asegura al usuario que se está comunicando con el servidor real (gracias a los certificados digitales).

Bibliografía

- **Cloudflare:** [Hypertext Transfer Protocol \(HTTP\)](#)
- **MDN Web Docs:** [An overview of HTTP](#)
- **AWS:** [Diferencia entre HTTP y HTTPS](#)
- **Complemento Visual:** [Video: Funcionamiento de HTTP \(YouTube\)](#)

